

Supplementary Information

LongPhase 2: uncertainty-aware co-phasing of small variants, structural variants and DNA methylation from long reads

Jyun-Hong Lin, [\[co-authors\]](#), Yao-Ting Huang

Contents

Supplementary Methods	2
Supplementary Notes	11
Supplementary Tables	12
Supplementary Figures	20

Supplementary Methods

These sections give the implementation-level detail of the modules that the main Methods describes only in outline, in enough detail for each algorithm to be reimplemented independently.

Supplementary Method 1. Allele-specific methylation calling (`modcall`)

Methods gives the genotyping criterion for a candidate CpG and the co-segregation criterion that a linked pair must satisfy. This section gives the parts of `modcall` that decide which candidates survive to the phasing graph.

Linking modes and the depth bound on the neighbour scan. In *SNV-anchored* mode, used whenever an SNV VCF was supplied, a candidate CpG was accepted as a strong anchor if it satisfied the co-segregation criterion of Methods against an SNV among the next 20 variants (`-connectAdjacent`). In *methylation-only* mode, used where no informative SNV is in range, which is where methylation is most valuable for bridging, the same criterion was applied between two candidate CpGs. In both modes a pair was considered only while the number of reads spanning it exceeded $\max(6, (n_1 + n_2)/4)$, where n_1 and n_2 are the numbers of reads recorded at the two sites; a pair falling to or below that bound terminates the neighbour scan for the current site rather than merely being skipped, so that support is never accumulated across a sparsely covered gap. Candidates never satisfying that criterion were held back as weak points and admitted only if they satisfied that criterion on at least six reads against an already accepted CpG, in an iterative expansion of two rounds (`-iterCount`, default 2).

Merging runs of CpGs. Runs of consecutive CpG coordinates are merged into a single locus represented by the first position. The reason is practical. 5mC is called on one strand at a time, so a CpG contributes methylated reads on the forward strand and unmethylated-looking reads on the reverse; merging consolidates both strands of a run into one locus carrying one read list, which removes the need for `phase` to reason about strand at all and lets methylation loci be voted on concurrently with SNVs by the unmodified graph algorithm. The resulting VCF records the methylated and unmethylated read names (`MR`, `NR`), the read strand and the per-site depths, and is passed to `phase` through `-mod-file`.

Supplementary Method 2. Read-level allele observations and copy-number interval detection

Per-class allele extraction. SNV alleles were read from the aligned base and inherit its Phred base quality. Insertion and deletion alleles were called from the CIGAR string: a read supports the ALT allele if a CIGAR insertion or deletion begins immediately after the variant position, and the REF allele otherwise; when a read deletion spans an SNV in a homopolymer of length ≥ 3 , the SNV allele was recovered from the base following the deletion. SV alleles were assigned by scanning a window of 20 CIGAR operations (`-svWindow`) around the SV start for an insertion or deletion whose length is within 10% of the SV length (`-svThreshold`); reads listed by the SV caller as supporting the SV were also assigned the ALT allele. 5mC alleles were taken from the `modcall` read lists (methylated = allele 0, unmethylated = allele 1) restricted to reads whose strand matched the recorded strand. Soft- and hard-clipping operations longer than 5 bp were counted per reference position, separately for clips at the front (left) and back (right) end of an alignment, as evidence of alignment breakpoints for copy-number detection (below).

The copy-number-aware filter (Methods; Supplementary Fig. 1e,f) needs the boundaries of segments that recruit reads from more than one locus. Those boundaries appear in the alignments as clusters of soft- and hard-clips, and `longphase` finds them with a scanner that walks the per-position clip counts of one chromosome in coordinate order.

Input. For every reference position at which at least one alignment carries a clipping operation longer than 5 bp, two counts are recorded: u , the number of alignments clipped at their front (left) end, and d , the number clipped at their back (right) end. Positions with no clip are absent from the map. A sentinel position 30 kb beyond the last observed position is appended before the scan and removed afterwards, so that an interval left open at the end of the chromosome is still closed.

State. The scanner is in one of three states: *idle*, *step* (a copy-number transition, entered on a strong front-clip cluster) and *ramp* (the progressively nested breakpoints left by breakage–fusion–bridge cycles, entered on a weak front-clip excess). In the two active states it carries a running excess c of front- over back-clips, a candidate start, a candidate end and three adaptive thresholds derived from the front-clip count n that opened or re-anchored the interval:

$$r = n, \quad p = \begin{cases} \lfloor n/2 \rfloor, & n \geq 10 \\ 5, & n < 10 \end{cases},$$

$$f = \begin{cases} 5, & n \geq 20 \\ \lfloor n/4 \rfloor, & 10 \leq n < 20 \\ 2, & n < 10, \end{cases}$$

where r is the re-anchor threshold, p the pull-down threshold and f the decay floor.

Transitions. With constants $\Pi = 5$ (push threshold), $A = 30,000$ bp (look-ahead), $E = 30$ (extend threshold), $\Lambda = 200,000$ bp (maximum interval length) and $\Sigma = 20$ (ramp commit threshold), the scanner at position x with counts (u, d) behaves as follows:

```

idle:
  if u >= PI and c == 0:
    enter STEP; c = u - d; start = x; end = x + A
    set r, p, f from u
  elif u > d and c == 0:
    enter RAMP; c = u - d; start = x; end = x + A

step:
  if u > r:                                # stronger breakpoint: re-anchor
    start = x; end = x + A; set r, p, f from u
  c += u - d
  if c > E: end = x + A
  if d >= p:                                # pull-down reached
    emit [start, x]; reset to idle
  elif c <= f and x <= end:                 # decayed to the floor in range
    emit [start, x]; reset to idle
  if x > end or c <= 0 or x - start >= LAMBDA:
    reset to idle                          # candidate discarded

ramp:
  if (c > SIGMA and d >= c/4) or (c <= SIGMA and d >= PI):
    emit [start, x]; reset to idle
  elif u >= PI:                             # promote to STEP
    c = u - d; start = x; end = x + A; set r, p, f from u
  else:
    c += u - d
    if c > E: end = x + A
    if x > end or c <= 0 or x - start >= LAMBDA:
      reset to idle                        # candidate discarded

```

Two behaviours are worth stating explicitly because they are not evident from the outline in the main Methods. First, a *step* interval is re-anchored, not merely extended, when a stronger front-clip

cluster than the one that opened it is met: the candidate start moves to the new position and the three thresholds are recomputed, so the interval reported is the one that begins at the strongest breakpoint. Second, the decay-floor exit applies only while the scan is still inside the look-ahead window ($x \leq \text{end}$); beyond it the interval is discarded rather than emitted, so a candidate that decays slowly over a long distance yields nothing.

Site removal. Within each emitted interval the mismatch-rate statistic $\text{MR}(v)$ defined in Methods is computed and heterozygous variants with $\text{MR}(v) \geq 0.7$ are removed from all reads. A variant is scored only when both allele populations have a non-zero mean load, so a site at which one population carries no other alternate allele is never removed on this basis.

Why MR separates the two cases. The statistic distinguishes the two situations of interest. At a genuine heterozygous site both allele populations are ordinary reads of the locus and carry comparable loads, so $\text{MR} \approx 0.5$. At a paralogue-specific difference the alternate allele is carried by reads that originate elsewhere and therefore mismatch the reference at many other sites in the same interval as well, which drives MR towards 1. A variant was scored only when both means were non-zero, so that sites at which one allele population carries no other alternate allele at all are never removed on this basis.

[Report, at each coverage, how many intervals the scanner emits genome-wide, their total span and the number of variants removed, and add the TP/FP histogram of $\text{MR}(v)$ with the 0.7 threshold marked as Supplementary Fig. 1g. None of the five constants above has been swept; if they were chosen by inspection, say so.]

Supplementary Method 3. Calibration of the voting and read-filtering constants

The constants in this section were set empirically. Writing the ambiguity of a pair as $s = \min(P, Q) / \max(P, Q)$, which ranges from 0 for unanimous support to 1 for an even split, the voting rule has three levels: a pair votes at weight $w = 20$ when it is near-certain, that is, when $s \leq \alpha = 0.1$ with total support ≥ 1 or, irrespective of s , when one pairing has accumulated less than 1 unit of support while the other has at least 1. The pair votes at unit weight otherwise, and does not vote at all when $s > \beta = 0.7$. Each of α , β , w , the base-quality threshold $\theta = 12$ and the single-read guard parameters ($N = 3$ votes, ambiguity cut $\mu = 0.2$) was fixed by a sweep on HG002 at 10–60 $\times$ before the experiments reported here, and none was re-tuned afterwards. [Add these sweeps as a Supplementary Figure (see Supplementary Fig. 9), and state here whether the tuning data are disjoint from the evaluation data reported in Figs. 2–6, or overlap them. This matters: five constants tuned on HG002 and then evaluated on HG002 is the same circularity objection the GNN faces, and a reviewer who notices it for one will apply it to the other. If the sweeps used the same HG002 alignments, say so plainly and argue why five scalar thresholds chosen from coarse grids do not constitute fitting.]

Supplementary Method 4. Graph neural network: features, architecture, labels, training and export

This section expands the outline given in Methods. Supplementary Tables 1 and 2 list the node and edge features with their exact definitions, Supplementary Table 3 the architecture and inference hyperparameters, Supplementary Table 4 the training configuration, and Supplementary Figs. 4–6 illustrate window construction and the layer.

Node features. Each allele node carried 31 features in four groups (Supplementary Tables 1 and 2; architecture in Supplementary Table 3 and Supplementary Fig. 6). *Phasing evidence*: PE; the allele’s genotype index; the reference and alternate genotype indices; whether the variant is the window trigger; whether it shares the trigger’s phase set; the allele index; the signed position offset from the trigger normalized by the largest absolute offset in the window; h_1 , h_2 , $h_1 + h_2$, the

vote ratio $\max(h_1, h_2)/(h_1 + h_2)$; $\log_2(1 + h_1 + h_2)$, $\ln(1 + h_1)$, $\ln(1 + h_2)$; and $\log_2$ of the total vote weight divided by its mean over the 21 variants centred on the variant. Here h_1 and h_2 were taken from the first outgoing DOT edge of each allele when one existed and from INFO/H1, INFO/H2 otherwise. (Feature indices, logarithm bases and the code order of the graph-structure features were verified against `GNNProcess.cpp:427–461` at commit `cc17fb1`; Supplementary Table 1 lists them by group.) *Genomic context* (computed from the reference when `-r` is given, and zero otherwise): homopolymer length at the variant (capped at 20); GC content in ± 50 bp; the longest tandem-repeat unit count with unit size 2–4 in ± 15 bp, divided by 10 and capped at 1; Shannon entropy of the ± 15 bp sequence; the number of variants within ± 500 bp, minus one and divided by 20; and $\log_{10}(1 + d)$ for the distance d to the nearest indel, capped at $\log_{10}(10001)$. *Graph structure*: the number of distinct target positions of the variant’s outgoing DOT edges, divided by 10 and capped at 1; $\ln(1 + n_{\text{PS}})/\ln(101)$ for the number n_{PS} of variants in the variant’s phase set; $\log_{10}(1 + |\text{offset}|)$ from the trigger; the standard deviation of the variant’s outgoing DOT edge weights, divided by 100 and capped at 1; and an indicator of whether the variant is a bridge vertex, i.e. whether deleting it increases the number of connected components of the position-level, undirected window graph. *Variant class*: four one-hot indicators (SNV, indel, SV, 5mC), the SNV/indel distinction being made on $||\text{ALT}| - |\text{REF}|| > 0$.

Edge features. Edges were taken from the DOT graph, restricted to pairs both of whose endpoints lie inside the window, symmetrized by copying the features of whichever direction was present, and augmented with a self-loop per node whose features are the mean of its non-self incident edges. Each edge carried six features: the logistic-transformed vote weight $1/(1 + e^{-w})$; $\log_{10}(1 + \text{genomic distance})$; $\ln(1 + w)$; whether the two endpoints carry the same allele index; whether the two variants belong to different phase sets; and w divided by the total outgoing weight of the source allele node within the window. [Earlier drafts glossed the fourth feature as “whether the two alleles are on the same haplotype”. It is an allele-index match, not a haplotype match (`GNNProcess.cpp:481`, `e.src_allele == e.dst_allele`); corrected above, and Supplementary Table 2 already states it correctly.] [Stale code comment, harmless but confusing to reimplementers: the header of `GNNModel.h` (lines 6–7 and 43) describes the edge tensor as `[N, N, 7]` and the per-layer edge encoder as `[7, 128]`, whereas `kEdgeFeat` is 6 (`GNNWeights.h:21`) and six features are what the code writes. Fix the comment.]

Architecture. The network follows the general, powerful, scalable (GPS) graph-transformer recipe¹. Node features were standardized with a batch-normalization layer whose statistics are frozen at inference, linearly projected to a 128-dimensional embedding and passed through a GELU non-linearity. Four identical layers then each combine a local and a global branch. The local branch is a four-head GATv2 message-passing layer^{2,3} in which the attention logit of head t for edge $j \rightarrow i$ is

$$\alpha_{ij}^{(t)} \propto \exp\left(\mathbf{a}_t^\top \text{LeakyReLU}_{0.2}([\mathbf{W}_r \mathbf{x}_i + \mathbf{W}_l \mathbf{x}_j + \mathbf{W}_e \mathbf{e}_{ij}]_t)\right), \quad (1)$$

where $[\cdot]_t$ selects the 32 coordinates of head t , $\mathbf{W}_e$ is a per-layer edge encoder without bias and the normalization is a softmax over the incoming edges of i (a node with no incoming edge instead receives the uniform average of $\mathbf{W}_l \mathbf{x}_j$ over all window nodes); the head outputs are concatenated and a bias is added. The global branch is four-head scaled dot-product self-attention over all nodes of the window⁴, with no masking. The two branch outputs are added to the residual stream, followed by layer normalization, a two-layer feed-forward block (128→256→128, GELU) with a second residual connection and layer normalization. The final node embedding is concatenated with two edge-context scalars, the sum and the non-negative maximum, over the node’s incoming edges excluding its self-loop, of the first edge feature, i.e. of the logistic-transformed vote weight $1/(1 + e^{-w})$ rather than of w itself. The pooled vector is then passed through a two-layer classifier (130→128→2) with softmax output.

The model comprises 687,358 float32 values (2,749,432 bytes), namely 124 in the input batch-normalization layer (scale, shift and the two frozen running statistics over 31 features), 4,096 in

the input projection, 166,528 in each of the four GPS layers, and 17,026 in the classifier. Inference is implemented in plain C++ with dense $N \times N$ adjacency and edge tensors and no external runtime dependencies; the exact erf-based GELU must be used, because the tanh approximation changes the output by $\sim 10^{-3}$ and does not reproduce the reference implementation. Predictions for the two allele nodes of a variant were averaged. Only variants in the central zone of each window were retained, that is those whose normalized absolute offset r from the trigger satisfies $r \leq \min(1, 10/\max(1, \text{round}(20r_{\max})))$, where $r_{\max}$ is the largest normalized absolute offset in the window; because offsets are normalized by that same largest offset, $r_{\max} = 1$ for every window containing a variant more than 1 bp from the trigger, so in practice the retained zone is the central half of the window’s genomic span. A variant covered by several windows received the running mean of its predictions. The same rule, with the same constant 10, defines which nodes carry a training loss, so the zone scored at inference is the zone the network was trained to score; nodes outside that zone are read as context but never scored. Because $r_{\max} \equiv 1$ in every window that contains a variant more than 1 bp from the trigger, the expression collapses to the constant 0.5 in practice and the $r_{\max}$ dependence is inert. [The criterion is written on the genomic-distance ratio r , but the form of the constant (`center_k/20`, with 20 the half-window in variants) suggests a variant-index criterion was intended (the central ± 10 variants of the ± 20 -variant window); the two coincide only under uniform variant spacing. Either restate the intent and keep the code, or replace both sides with the index form and re-run; do not leave the dead $r_{\max}$ term in the published description.]

Training augmentations. Two label-preserving augmentations were applied to training windows only, each independently with probability 0.5: a haplotype swap, which exchanges the two allele nodes of every variant and complements the HP feature, an operation the phasing problem is invariant under; and a coverage down-scaling, which multiplies h_1 , h_2 , their sum, the outgoing-weight standard deviation and the raw edge weights by a factor drawn uniformly from $[0.4, 0.7]$ and recomputes the derived logarithmic features, applied only to windows containing at least one error node.

Labels. Labels were derived from the v5.0q benchmark restricted to chr1–chr22 small variants, the same truth set used for evaluation, and describe the phase of a variant relative to its neighbours rather than in absolute terms. Within each window the variants were grouped by phase set; in each group the first variant present in the benchmark fixed the orientation of that group and was labelled *correct*, and every later variant was labelled *correct* if its allele-to-haplotype orientation agreed with that anchor and *disagreeing* otherwise. Maximal runs of consecutive disagreeing variants were then resolved so that a run is charged to the event that created the run, not to each member. A run of length one became *unphase* if that variant’s PE was at least 0.9 and *flip* otherwise, whereas a run of length two or more had its first variant, the switch point, relabelled *break*, and all remaining members reset to *correct*. Variants absent from the benchmark, and all SV and 5mC records, for which the benchmark carries no phase truth, were left unlabelled: they are masked from the loss but remain in the graph as context, which is why a four-class window can be scored at all. The released model was trained in binary mode, in which *flip*, *unphase* and *break* are merged into a single *error* class; this matches the single action the module is allowed to take. Only labelled nodes inside the central zone of a window contribute to the loss, under the same `center_k = 10` rule used at inference.

Class balance and loss. Among the labelled central nodes of the training split, 17,812,710 (97.04%) were *correct* and 544,170 (2.96%) *error*, and 21.5% of windows contained at least one error node; the prevalence is far above the genome-wide switch-error rate because windows are opened only at high-entropy triggers and because the label is assigned relative to a window-local anchor rather than to a whole phase block. The imbalance is handled by subsampling, not by re-weighting the full distribution. Each minibatch keeps all of its error nodes and draws correct nodes uniformly at random to at most four times that number (`-ratio_corr 4`), so the gradient

is taken on an approximately 1:4 subset, and because the draw is independent at every step the network still sees the whole set of correct nodes over training. The cross-entropy on that subset is focal-weighted⁵ by $(1 - p_t)^\gamma$ with $\gamma = 1.5$ and uses label smoothing of 0.05. A differentiable surrogate for the selection metric below is implemented in the training script but was disabled for the released model ($\alpha = 0$), so the reported weights were trained on the focal loss alone.

Model selection. Checkpoints were selected on the validation chromosomes by a net-gain criterion instead of validation loss or F1. At each epoch the predicted error probability was thresholded at each of 81 values spanning $[0.01, 0.99]$ and the score

$$S = R \left(2 - \frac{1}{P} \right) \quad (2)$$

was maximized over those thresholds subject to a recall of at least 0.005, where P and R are the precision and recall of the error class; S is positive only for $P > 0.5$ and is the net fraction of true errors removed under the accounting in which a detection is worth one error and a false detection costs one, so epochs with $P \leq 0.5$ were instead ordered by $P - 1$. The best checkpoint was epoch 44, with $S = 0.0235$ at $P = 0.626$, $R = 0.058$ and a threshold of 0.868; no later epoch improved on that checkpoint before early stopping. On the held-out chromosomes chr17, chr21 and chr22 that checkpoint attained an accuracy of 0.941 with error-class precision, recall and F1 of 0.258, 0.499 and 0.340 under the argmax decision, and 0.946 with 0.273, 0.468 and 0.345 at the decision threshold tuned on the validation chromosomes, over 41,378 labelled error nodes. The threshold applied by `gnn`, 0.30, is far more permissive than either threshold, because the module’s only action is to withhold phase: a false detection costs one phased variant rather than one switch error, so the break-even precision built into S governs checkpoint ranking but not deployment. [State how 0.30 was chosen: a sweep, on which data and against which objective, or an a-priori choice. Add the phased-fraction/switch-error trade-off across thresholds as a supplementary panel.]

Export and numerical agreement. The released weights are the checkpoint `model_e6.pt`, exported as `model_e6.onnx` and recorded by that name in the header of the generated weight file (`GNNWeights.h`, lines 3–4 at commit cc17fb1); the suffix denotes the six edge features of this model generation, not a training epoch. `export_onnx_2.py` re-implements the trained network with dense $N \times N$ adjacency and edge tensors in place of the sparse PyTorch Geometric operators, transfers the trained tensors (splitting the fused attention projection into separate Q, K and V matrices), and exports to ONNX at opset 17 with N as a dynamic axis. The logistic transform of the vote weight and the self-loop features are folded into the input convention at this point, so the exported graph and the C++ implementation both receive edge features with those already applied. `embed_weights.py` then reads the ONNX initializers, recovers the architecture dimensions from the tensor shapes, and writes them together with a base64-encoded little-endian float32 blob in a fixed tensor order as `GNNWeights.h`; the C++ forward pass in `GNNModel.h` reads the blob in that order and refuses to run if its length disagrees with `kParamCount`. Agreement was checked at each link of the chain: the dense rewrite against the PyTorch Geometric model, and the ONNX graph against the dense rewrite, on a real training window, with an acceptance bound of 10^{-3} on the absolute difference in predicted error probability; and the compiled C++ against ONNX Runtime on six synthetic graphs of 3–120 nodes at edge densities 0.1–0.4, including one graph with a single edge and no other incoming edges, where the largest absolute difference was 3×10^{-7} against an acceptance bound of 10^{-5} . [Report the achieved maximum differences for the first two links as well; `export_onnx_2.py` prints both, but the export log was not retained.]

Supplementary Method 5. Read haplotagging

`longphase haplotag` assigns each alignment to a haplotype using the phased VCFs produced by `phase` (and, optionally, corrected by `gnn`). The main Methods gives the decision rule; this section gives the evidence model, which differs from the one used inside `phase` in ways that matter to anyone comparing haplotagged fractions between tools.

Alignment selection. Primary alignments with mapping quality ≥ 1 (`-q`, `-qualityThreshold`) are considered. Supplementary alignments are tagged only under `-tagSupplementary`; secondary alignments never are. Any pre-existing HP, PS and PQ tags are removed from every record before tagging, so re-tagging an already-tagged file is safe. Unmapped records are passed through unchanged.

Evidence collection. Each alignment is traversed once along its CIGAR, and four classes of evidence are collected into two per-haplotype tallies (n_1, n_2) :

- *SNVs* are read from `M`, `=` and `X` operations; a read carrying the allele phased to haplotype k adds 1 to n_k .
- *Small indels* are scored by allele length rather than by sequence comparison. Writing ℓ_1 and ℓ_2 for the lengths of the two phased alleles at the site, a read that carries an indel there supports the haplotype whose allele has $\ell \neq 1$, and a read that does not carries support for the haplotype whose allele has $\ell = 1$. When both or neither allele has length 1 the site casts no vote.
- *SVs* are re-derived from the alignment, not taken from the caller's read list: a window of $\pm$ `-svWindow` CIGAR operations (default 20) around the current operation is scanned for an insertion or deletion whose length is within `-svThreshold` (default 0.10) of the SV length, and the first match adds 1 to the haplotype the SV is phased to.
- *Allele-specific 5mC* sites are matched by read name, not by position: the methylated-read list `INFO/MR` of the phased 5mC VCF adds 1 to the haplotype that record is phased to, for each read it names. The unmethylated list `NR` is not consulted, so a 5mC site contributes evidence only to the reads that are methylated at it.

Every informative observation contributes exactly 1, whatever its class. This is deliberately different from the read-based correction inside `phase`, where SNV and SV alleles contribute 1, indel and tandem-repeat-indel alleles 0.1 and 5mC alleles nothing (Methods). A read can therefore be tagged in `haplotag` on indel and methylation evidence that would have been discounted during phasing.

Decision and phasing quality. Let $m = \max(n_1, n_2)$ and $\mu = \min(n_1, n_2)$. The read is assigned to the majority haplotype when $m/(m + \mu) \geq$ `-p`, `-percentageThreshold` (default 0.6) and is otherwise left untagged. A Phred-scaled phasing quality is computed for every alignment as

$$\text{PQ} = \begin{cases} 0 & m = 0, \\ 40 & \mu = 0, \\ \left\lfloor -10 \log_{10} \frac{\mu}{m + \mu} \right\rfloor & \text{otherwise,} \end{cases} \quad (3)$$

so a read whose alleles are unanimous receives the ceiling value 40 and a read split 2:1 receives 4. PQ is written as an integer auxiliary tag `PQ:i` alongside `HP:i` and `PS:i`, and only on tagged reads; untagged reads carry none of the three. PQ is not reported in the main text and no analysis in this paper uses it.

Phase-set veto. The phase sets of all informative variants on the read are counted. If more than one distinct PS is represented, the read is left untagged regardless of how one-sided its haplotype tally is, because a majority computed across two independent phase sets has no meaning. This veto bounds the taggable fraction from above and is the main reason a read spanning a block boundary is not tagged. The PS value written on a tagged read is that of the first phased variant at or after the alignment start.

[Three items for this module. (i) Confirm that the PS written is always the phase set of the read's own informative variants; it is read from the first variant at or after the alignment start,

which coincides with the read’s phase set only because the multi-PS veto has already fired, and the intent should be stated or the code made explicit. (ii) Report the fraction of reads left untagged by the phase-set veto separately from those failing the 0.6 majority, at each coverage; the two have different causes and only the second is a confidence statement. (iii) Decide whether PQ should be documented as a supported output, in which case it belongs in the README and in the VCF/BAM header description, or removed.]

Supplementary Method 6. Evaluation metrics and region restriction

Metrics were computed with `longphase compare` (Supplementary Fig. 8), a C++ re-implementation of the `whatshap compare` algorithm⁶. The first VCF on the command line is the truth set and every further VCF is a query, so several tools can be scored against one truth set in a single invocation and reported side by side.

Block intersection. For each chromosome, heterozygous records common to the truth and a query are intersected by position and by REF/ALT alleles. The intersection is partitioned into maximal runs that lie in one truth phase set and one query phase set simultaneously; runs of length one carry no phase information and are discarded. Only variants inside the surviving blocks enter any metric, so the denominators of the switch error rate and the Hamming distance are the same for every tool scored in the same run.

Switch errors and flips. Each block yields two binary strings, the truth haplotype assignment p_0 and the query assignment q_0 of its variants. Both are converted to *switch encoding*, the string of relative changes between consecutive positions, which removes the arbitrary orientation of the block. Switch errors are the Hamming distance between the two switch-encoded strings. The decomposition into isolated switches and flips follows WhatsHap’s `compute_switch_flips`: a maximal run of k consecutive disagreements in switch space is charged as $\lfloor k/2 \rfloor$ flips and $k \bmod 2$ switches, so that a single variant assigned to the wrong haplotype counts as one flip rather than two switches. The *switch error rate* is the total number of switch errors divided by the number of assessed consecutive pairs.

Hamming distance. For each block the query strings (q_0, q_1) are compared with the truth strings (t_0, t_1) in both orientations, and the smaller of $H(t_0, q_0) + H(t_1, q_1)$ and $H(t_0, q_1) + H(t_1, q_0)$ is halved to give the number of misassigned variants in that block. Block values are summed and expressed as a percentage of compared variants.

Block N50. Block spans in bp are sorted in decreasing order and the N50 is the span at which the cumulative sum first reaches half the total, which is the span-weighted median. It is computed twice: once per chromosome, and once over the block spans of all chromosomes pooled. The values plotted in Figs. 2d, 3d and 5 are the pooled, genome-wide N50 of the query, which does not depend on the truth set.

Region restriction. Comparison can be restricted to a set of intervals, either inline with `-regions` (comma-separated `chrN`, `chrN:start-end` or `chrN:pos`, 1-based inclusive) or from a file with `-regions-bed` (0-based half-open); the two can be combined, and their union is used. This is the mechanism by which a benchmark high-confidence BED is applied. With `-only-snvs` all non-SNV records are dropped before intersection, which is how the cross-tool comparisons in this paper were restricted to SNVs so that co-phasing additional classes does not change the evaluated set. With `-sw-bed` the position of every switch error is written as a BED record carrying the error type and the truth and query phase sets, which is the input of the unphased-variant analysis in Fig. 6. `-sample` selects the sample column and `-ignore-sample-name` accepts single-sample VCFs whose sample names differ.

[Report the concordance of `longphase compare` with `whatshap compare` explicitly: identical switch-error and Hamming counts on all autosomes for all four tools at one coverage, with the `whatshap compare` version used, in a supplementary table. State whether the results in this paper were produced with a region restriction and, if so, which BED.]

Supplementary Notes

Supplementary Note 1. Reading the phasing graph export

With `-dot`, `longphase phase` writes one file per chromosome, `<prefix>.<chrom>.dot`, containing one line per voting pair, `P.a -> Q.b [label=w]`, where `P` and `Q` are 1-based positions, `a`, `b` $\in \{1, 2\}$ index the reference (1) and alternate (2) allele, and `w` is the weighted haplotype vote accumulated at `P` for that allele at the time the vote was cast. The companion VCF carries, for every phased variant, the fields `INFO/H1`, `INFO/H2` (weighted votes for haplotype 1 and 2) and `INFO/PE` (their normalized Shannon entropy). The `gnn` sub-command reads both files; users who wish to inspect why a variant was unphased can therefore reconstruct its local window without re-running the phaser.

Supplementary Note 2. Reproducing the demonstration

The repository ships a 500-kb demonstration (`chr1:121.5–122.0 Mb`; `demo/demo.sh`) that runs `modcall`, SNV-only and four-class `phase`, `gnn` correction of both outputs and `compare` against a truth VCF, and prints switch errors, phased SNV counts and Hamming distance for the four configurations. Total runtime is under two minutes on four threads.

Supplementary Tables

Supplementary Table 1: Node features of the graph neural network. Each allele node of a window carries 31 features; # is the 1-based index in the feature vector (`GNNProcess.cpp`, `f[0]–f[30]`), listed here by group, not by index. h_1, h_2 : weighted haplotype votes (taken from the DOT edge labels when available, otherwise from INFO/H1, INFO/H2); PE: phasing entropy; PS: phase set; “centre” is the trigger variant of the window. Genomic-context features are computed from the reference when `-r` is given and are zero otherwise.

#	Feature	Definition
<i>Phasing evidence (16)</i>		
1	PE	normalized Shannon entropy of (h_1, h_2) , 0–1 bit
2	HP	haplotype of this allele node (genotype index)
3	gt_ref	genotype index of the reference allele (0)
4	gt_alt	genotype index of the alternate allele (1)
5	is_center	1 if the variant is the window trigger
6	is_same_ps	1 if the variant shares the trigger’s PS
7	allele	0 for the reference-allele node, 1 for the alternate-allele node
8	rel_offset	$(\text{pos} - \text{pos}_c) / \max \text{pos} - \text{pos}_c $ over the window, signed
9	H1	h_1
10	H2	h_2
11	total_votes	$h_1 + h_2$
12	vote_ratio	$\max(h_1, h_2) / (h_1 + h_2)$
13	log_total	$\log_2(1 + h_1 + h_2)$
14	log_h1	$\ln(1 + h_1)$
15	log_h2	$\ln(1 + h_2)$
25	rel_vote_depth	$\log_2((h_1 + h_2) / \overline{(h_1 + h_2)}_{\pm 10 \text{ variants}})$
<i>Genomic context (6)</i>		
16	hp_len	homopolymer run length at the variant (capped at 20)
17	gc_content	GC fraction in ± 50 bp
18	str_context	longest tandem-repeat unit count (unit 2–4 bp) in ± 15 bp, /10, capped at 1
19	seq_entropy	Shannon entropy of the ± 15 bp sequence (bits)
20	neigh_dens	$(\text{number of variants within } \pm 500 \text{ bp} - 1) / 20$
24	dist_near_indel	$\log_{10}(1 + \text{distance to nearest indel})$, capped at $\log_{10}(10001)$
<i>Graph structure (5)</i>		
21	out_degree	number of distinct target positions of outgoing DOT edges, /10, capped at 1
22	ps_block_size	$\ln(1 + \text{variants in the PS}) / \ln(101)$
23	log_center_dist	$\log_{10}(1 + \text{pos} - \text{pos}_c)$
26	out_weight_std	s.d. of outgoing DOT edge weights, /100, capped at 1
27	is_bridge	1 if removing the variant disconnects the window graph
<i>Variant class (4, one-hot)</i>		
28–31	is_SNV, is_indel, is_SV, is_5mC	variant class indicators

Supplementary Table 2: Edge features of the graph neural network. Edges are taken from the DOT export, restricted to node pairs inside the window, symmetrized, and complemented by one self-loop per node whose features are the mean over the node’s incident edges. w is the DOT edge label (accumulated vote weight).

#	Feature	Definition
1	weight	$1/(1 + e^{-w})$
2	log_dist	$\log_{10}(1 + \text{pos}_{\text{dst}} - \text{pos}_{\text{src}})$
3	log_weight	$\ln(1 + w)$
4	is_same_hp	1 if the two allele nodes have the same allele index
5	is_inter_ps	1 if the two variants belong to different phase sets
6	weight_ratio	$w / (\text{sum of outgoing edge weights of the source allele node})$

Supplementary Table 3: Architecture and inference hyperparameters of the graph neural network.

Component	Setting
Input normalization	BatchNorm over 31 node features (frozen statistics at inference)
Input projection	Linear $31 \rightarrow 128$, GELU
Layers	4 GPS layers, hidden size 128
Local branch	GATv2, 4 heads $\times$ 32, LeakyReLU slope 0.2, per-layer edge encoder $6 \rightarrow 128$
Global branch	multi-head self-attention, 4 heads, Q/K/V/O projections $128 \rightarrow 128$
Fusion	$x \leftarrow x + \text{local} + \text{global}$; LayerNorm ($\varepsilon = 10^{-5}$)
Feed-forward	$128 \rightarrow 256 \rightarrow 128$, GELU, residual, LayerNorm
Classifier	concat $[h_i, \sum_j \sigma(w_{ij}), \max_j \sigma(w_{ij})]$ $(130) \rightarrow 128$ (GELU) $\rightarrow 2$, softmax; σ logistic, self-loop excluded
Parameters	687,358 (float32, 2.75 MB, embedded in the binary)
Window	trigger PE ≥ 0.8 ; ± 20 variants (≤ 41 variants, ≤ 82 allele nodes; a 256-node cap exists but is never reached at this window size)
Decision	unphase if mean $P(\text{error}) \geq 0.30$ over the two allele nodes and all covering windows
Post-processing	split disconnected phase sets (largest component keeps PS); bridge protection off
Training	see Supplementary Table 4

Supplementary Table 4: Training configuration of the graph neural network. Values are those recorded in the released checkpoint `model_e6.pt` and in `train_e6.log`; every setting is the default of `train_gnn_29.py` except `-binary`, which was passed explicitly. “Scored nodes” are labelled allele nodes inside the central zone of a window, the only nodes that contribute to the loss.

Item	Setting
<i>Data</i>	
Source runs	HG002 ONT R10.4.1, four-class co-phasing, 10/12/14/16/18/20 × × 10 down-sampling replicates (60 runs)
Window extraction	<code>prepare_gnn_data_10.py</code> , <code>-pe_threshold 0.8</code> , <code>-window 20</code> , GRCh38 no-alt analysis set
Truth set	v5.0q benchmark, chr1–chr22 small variants
Split (by chromosome)	train chr1–chr14, chr18–chr20; validation chr15, chr16; test chr17, chr21, chr22
Windows / scored nodes	train 602,788 / 18,356,880; val 67,880 / 1,509,576; test 51,448 / 1,367,580
Class prevalence (train)	correct 17,812,710 (97.04%), error 544,170 (2.96%); 21.5% of windows contain an error node
Labels	anchor-relative within each PS; single disagreement → <i>unphase</i> if $PE \geq 0.9$ else <i>flip</i> ; run of ≥ 2 → first variant <i>break</i> , rest <i>correct</i> ; merged to one <i>error</i> class
Masked from loss	variants absent from the benchmark; all SV and 5mC records (the truth set holds small variants only); nodes outside the central zone
<i>Optimization</i>	
Loss	focal cross-entropy, $\gamma = 1.5$, label smoothing 0.05, on a per-batch subset of all error nodes plus $\leq 4 \times$ as many randomly drawn correct nodes
Optimizer	AdamW, $\beta = (0.9, 0.999)$, $\varepsilon = 10^{-8}$, weight decay 10^{-3}
Learning rate	3×10^{-4} , no warm-up, cosine annealing to 10^{-5} over 100 epochs
Batch size	64 windows, batched as disjoint graphs
Regularization	dropout 0.4 (local branch, global branch, FFN; inactive at inference); gradient-norm clipping 1.0
Graph preprocessing	reverse edge added per DOT edge with identical features (undirected)
Augmentation (train only)	haplotype swap, $p = 0.5$; coverage down-scaling $\times U[0.4, 0.7]$, $p = 0.5$, error-bearing windows only
Epochs	budget 100, early stopping after 25 epochs without improvement; stopped at 69
Seed	42 (Python and PyTorch generators)
Environment	PyTorch 2.11.0, PyTorch Geometric 2.7.0, Python 3.12, CUDA 13.0, cuDNN 9.19, one NVIDIA GeForce RTX 5090
<i>Selection and export</i>	
Selection criterion	max over 81 thresholds in $[0.01, 0.99]$ of $R(2 - 1/P)$ subject to $R \geq 0.005$, on chr15–chr16; $P - 1$ where $P \leq 0.5$
Selected checkpoint	epoch 44; $S = 0.0235$ at $P = 0.626$, $R = 0.058$, threshold 0.868
Held-out test (chr17, 21, 22)	argmax: accuracy 0.941, error-class $P/R/F_1 = 0.258/0.499/0.340$; tuned threshold: 0.946, 0.273/0.468/0.345; $n = 41,378$ error nodes
Deployment threshold	0.30, more permissive than the selection threshold because the only action is to withhold phase
Export	ONNX opset 17, dynamic N ; base64 float32 blob compiled into <code>GNNWeights.h</code> ; C++ vs. ONNX Runtime max. absolute difference 3×10^{-7}

Supplementary Table 5: Parameters of longphase phase and their defaults.

Parameter	Default	Meaning
<code>-ont / -pb</code>	–	sequencing platform (required; <code>-ont</code> enables the homopolymer SNV filter)
<code>-indels</code>	off	phase small indels
<code>-indelQuality</code>	0	exclude indels with QUAL below this value
<code>-sv-file, -mod-file</code>	–	SV VCF (Sniffles2/cuteSV with read names); 5mC VCF from modcall
<code>-q, -mappingQuality</code>	1	minimum mapping quality of alignments
<code>-x, -mismatchRate</code>	3	mark reads as unreliable above this mismatch rate
<code>-p, -baseQuality</code>	12	base quality below which an observation is down-weighted
<code>-e, -edgeWeight</code>	0.1	weight of a down-weighted observation (otherwise 1)
<code>-a, -connectAdjacent</code>	35	number of downstream variants each variant is linked to
<code>-d, -distance</code>	300,000	a variant is skipped (no votes, no haplotype) when the next heterozygous variant is further away than this; edges and votes themselves are not distance-limited
<code>-l, -edgeThreshold</code>	0.7	pair casts no vote if $\min(P, Q) / \max(P, Q)$ exceeds this (0.3 for SNV–5mC)
<code>-L, -overlapThreshold</code>	0.2	overlap fraction above which the shorter of two alignments of a read is dropped
<code>-w, -svWindow</code>	20	CIGAR operations scanned around an SV start (the help text prints <code>-sv-window</code> , which does not parse)
<code>-h, -svThreshold</code>	0.10	relative length tolerance for a CIGAR indel to support an SV (help text: <code>-sv-threshold</code>)
<code>-m, -readConfidence</code>	0.65	minimum majority fraction to tag a read in read-based correction
<code>-n, -snpConfidence</code>	0.75	minimum confidence to keep a variant’s phase after read-based correction
<code>-dot</code>	off	export the phasing graph (required by <code>gnn</code>)

Supplementary Table 6: Parameters of `longphase modcall` and their defaults. Values are the compiled-in defaults at commit `cc17fb1`; the on-line help text advertises 0.7 for `-heterRatio` and 6 for `-connectAdjacent` and the long option for the BAM input is `-methylbamfile` ([\[fix the help text and confirm the values used for the benchmarks\]](#)).

Parameter	Default	Meaning
<code>-m, -modThreshold</code>	0.8	ML probability at or above which a base is methylated
<code>-u, -unModThreshold</code>	0.2	ML probability at or below which a base is unmethylated
<code>-e, -heterRatio</code>	0.6	minimum $\min(m, u) / \max(m, u)$ for a heterozygous site
<code>-i, -noiseRatio</code>	0.2	maximum fraction of reads with intermediate probability
<code>-a, -connectAdjacent</code>	20	neighbouring variants searched for SNV co-segregation
<code>-c, -connectConfidence</code>	0.9	minimum majority ratio of co-segregating reads
<code>-all</code>	off	also output homozygous modification sites

Supplementary Table 7: Parameters of `longphase gnn` and `longphase haplotag`.

Parameter	Default	Meaning
<i>gnn</i>		
<code>-s, -snp-file</code>	–	phased SNV/indel VCF from <code>phase</code>
<code>-sv-file, -mod-file</code>	–	phased SV and 5mC VCFs (corrected copies are written)
<code>-dot-prefix</code>	from <code>-s</code>	prefix of the DOT files
<code>-r, -reference</code>	–	reference FASTA (enables genomic-context features)
<code>-pe-threshold</code>	0.8	phasing entropy that opens a window
<code>-window</code>	20	variants on each side of the trigger
<code>-B, -break-threshold</code>	0.30	error probability at or above which a variant is unphased
<code>-respect-bridge</code>	off	never unphase bridge vertices
<code>-no-split-blocks</code>	off	keep the original PS even if the block becomes disconnected
<i>haplotag</i>		
<code>-q, -qualityThreshold</code>	1	minimum mapping quality of tagged alignments
<code>-p, -percentageThreshold</code>	0.6	minimum fraction of a read’s informative alleles on the majority haplotype
<code>-tagSupplementary</code>	off	also tag supplementary alignments
<code>-sv-file, -mod-file</code>	–	use phased SVs and 5mC sites as additional tagging evidence
<code>-w, -svWindow</code>	20	CIGAR operations scanned around an SV start (help text: <code>-sv-window</code>)
<code>-h, -svThreshold</code>	0.10	relative length tolerance for a CIGAR indel to support an SV (help text: <code>-sv-threshold</code>)
<code>-region</code>	all	restrict tagging to <code>chrom</code> , <code>chrom:start</code> or <code>chrom:start-end</code>
<code>-log</code>	off	write a per-read record of the tagging decision
<code>-cram</code>	off	write CRAM instead of BAM

Supplementary Table 8: Parameters of `longphase compare`. The first VCF on the command line is the truth set and every further VCF is a query, so several tools can be scored against one truth set in a single run.

Parameter	Default	Meaning
<code>-o, -out-prefix</code>	<code>compare_result</code>	prefix of the output TSV report
<code>-t, -threads</code>	1	threads for the per-block switch and Hamming computation
<code>-n, -names</code>	–	data-set names used in the report, in the order of the VCFs
<code>-s, -sample</code>	first in VCF	sample column to compare
<code>-ignore-sample-name</code>	off	accept single-sample VCFs whose sample names differ
<code>-only-snvs</code>	off	drop all non-SNV records before block intersection
<code>-sw-bed</code>	–	write one BED record per switch error (chrom, start, end, type, truth PS, query PS, data-set pair)
<code>-regions</code>	all	restrict the comparison to <code>chrN</code> , <code>chrN:start-end</code> or <code>chrN:pos</code> , 1-based inclusive, comma separated
<code>-regions-bed</code>	all	restrict the comparison to the intervals of a BED file, 0-based half-open; combines with <code>-regions</code>

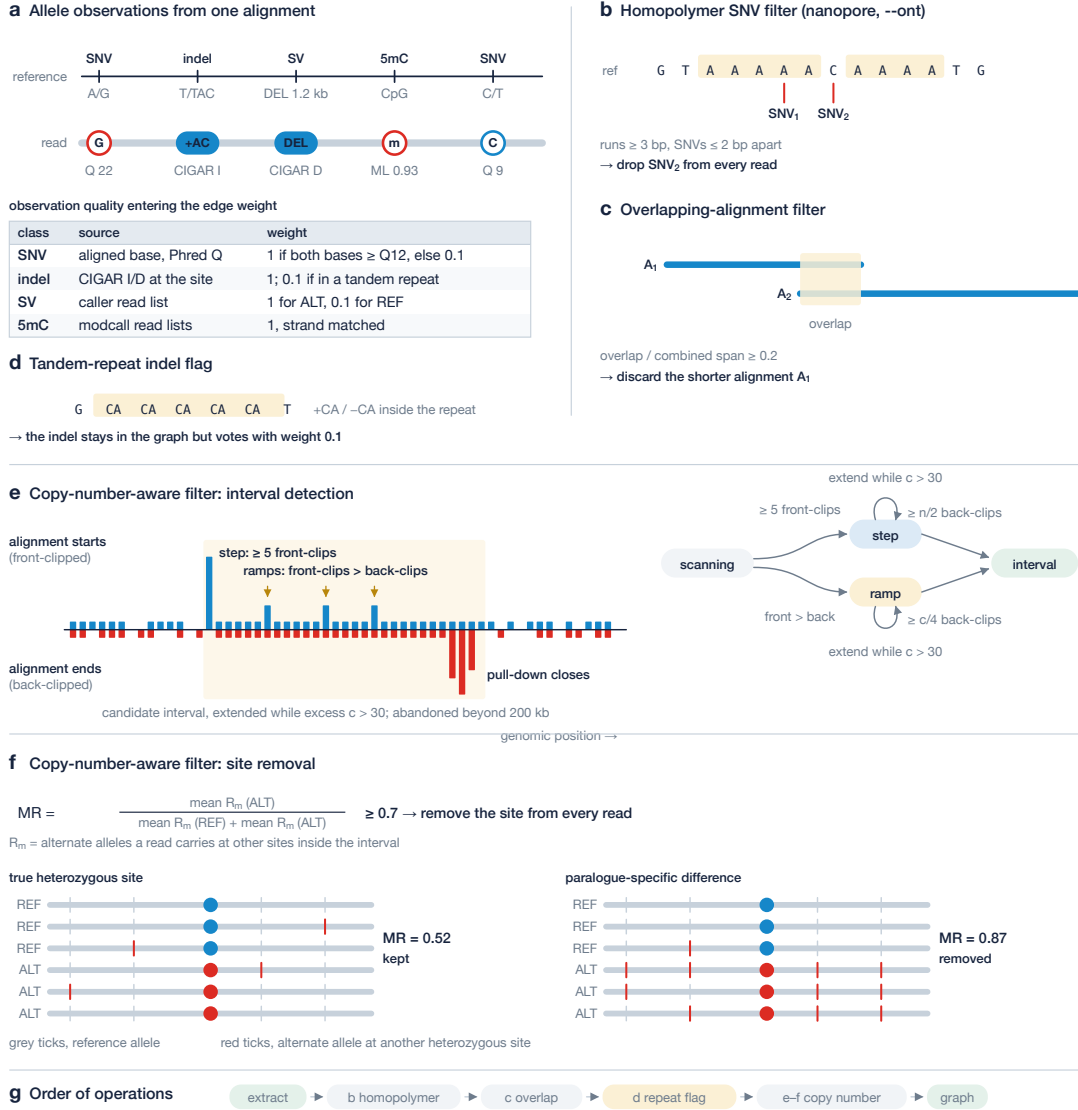
Supplementary Table 9: Software and data used in the benchmarks. [Complete versions and accessions.]

Item	Version	Notes
HG002 nanopore R10.4.1 reads	[-]	[flow cell , kit , basecaller model , source]
Reference	GRCh38 [build]	
minimap2	[-]	<code>-ax map-ont -y</code>
Clair3	[-]	SNV and indel calls
Sniffles2	[-]	<code>-output-rnames</code>
GIAB v4.2.1 HG002	v4.2.1	GRCh38-based small-variant benchmark
T2T-HG002 v1.1	GIAB v5 draft	projected onto GRCh38; referred to as v5.0q
small-variant benchmark ^{7,8}		
WhatsHap	2.8	[options]
HapCUT2	1.3.4	default parameters
LongPhase 1.0	1.0.x [exact]	<code>-ont</code>
LongPhase 2	[release]	<code>-ont</code> ; <code>gnn</code> with defaults

Supplementary Table 10: Selected published tools, pipelines and studies that use LongPhase.
 Compiled from Europe PMC (80 indexed citing articles) and OpenAlex (96 citing works) for 10.1093/bioinformatics/btac058, September 2026; the “use” column quotes or paraphrases the citing article’s Methods.

Work	Journal, year	Use of LongPhase
<i>Variant callers and pipelines</i>		
ClairS ⁹	Nat. Methods 2026	default germline phasing of tumour and normal alignments (<code>-whatshap_for_phasing</code> the alternative); haplotagging selectable
ClairS-TO ¹⁰	Nat. Commun. 2025	default intermediate phasing and haplotagging of the tumour alignment (WhatsHap selectable)
Clair3-RNA ¹¹	Nat. Commun. 2025	phasing in the long-read RNA pipeline
Clair3 (accelerated) ¹²	Bioinformatics 2026	selectable intermediate phasing and haplotagging (<code>-longphase_for_phasing</code> ; WhatsHap the default)
DeepVariant local haplotagging ¹³	Nat. Commun. 2024	not used; cited as one of the haplotaggers (with WhatsHap and Margin) that Clair3 and Medaka employ
Severus ¹⁴	Nat. Biotechnol. 2025	one of the documented routes to the haplotagged BAMs it consumes (WhatsHap the other)
EPI2ME wf-human-variation ¹⁵	ONT workflow	intermediate per-contig phasing (<code>phase_contig</code>); WhatsHap for final phasing and haplotagging
HiFi-somatic-WDL ¹⁶ cuteHap ¹⁷	PacBio workflow Adv. Sci. 2026	optional phasing tool read phasing before haplotype-aware SV calling
SpecImmune ¹⁸	Adv. Sci. 2026	joint SNV–SV phasing of immune genes
<i>Biological and clinical studies</i>		
Fluctuating DNA methylation in cancer ¹⁹	Nature 2025	read phasing (v1.7) and <code>modcall</code> for CpG methylation
Advanced cancer cohort (POG) ²⁰	Cell Genomics 2024	discussed as phasing method for tumours
Pediatric epilepsy episignatures ²¹	Nat. Commun. 2024	phasing of Clair3 and SV calls
GNAS transcripts in PHP1B ²² 3-hour genome for neonatal risk ²³	PNAS 2025 Genet. Med. Open 2024	phasing of Clair3 calls real-time phasing during sequencing
Ultrarapid diagnostics in critical care ²⁴	Eur. J. Hum. Genet. 2025	read phasing (v1.0.7)
HYDIN in primary ciliary dyskinesia ²⁵	Eur. J. Hum. Genet. 2024	phasing of Clair3 calls
Antisense oligonucleotides in APBD ²⁶	Nucleic Acids Res. 2025	phasing of Clair3 calls
Native CpG methylation fine-mapping ²⁷	HGG Adv. 2026	phasing and haplotagging (chosen for speed and SV support)
Canadian national platform benchmark ²⁸	Front. Genet. 2026	phasing inside the Clair3 workflow
Pallas’s cat genome ²⁹	NAR Genom. Bioinform. 2023	haplotagging for allele-specific methylation
<i>Methods evaluated against LongPhase</i>		
HiPhase ³⁰	Bioinformatics 2024	comparison method
Ralphi ³¹	Genome Res. 2025	comparison method (fastest runtime on ONT/PacBio)
GCphase ³²	BMC	comparison method

Supplementary Figures

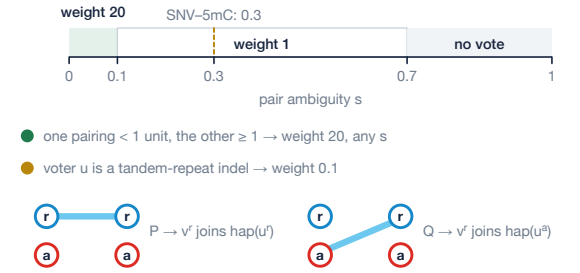


Supplementary Fig. 1: Read-level allele observations and pre-graph filters. **a**, For every alignment, each overlapped variant yields an allele observation with an associated quality: SNVs from the aligned base (Phred quality), indels from CIGAR operations starting at the site, SVs from CIGAR operations of matching length or from the caller's read list, and 5mC from the *modcall* read lists. The table summarizes how each class enters the edge weights. **b**, Homopolymer SNV filter, applied under *-ont*: when two SNVs both lie in homopolymers of length ≥ 3 and are ≤ 2 bp apart, the downstream SNV is removed. **c**, Overlapping-alignment filter for supplementary alignments of one read. **d**, Indels in tandem repeats are flagged and vote with weight 0.1. **e**, Copy-number-aware filter, interval detection. Per-position counts of alignments starting (front-clipped, above the axis) and ending (back-clipped, below) are scanned by a two-state detector (right): a *step* boundary (≥ 5 front-clips at one position) marks a copy-number transition; a *ramp* boundary (front-clips exceeding back-clips without reaching that threshold) marks the nested breakpoints left by breakage–fusion–bridge cycles. Both states carry a running excess c of front- over back-clips and extend the candidate end by a 30 kb look-ahead while $c > 30$; a step closes at an adaptive pull-down threshold ($n/2$ for an opening front-clip count $n \geq 10$, else 5) or when c decays to its floor, a ramp closes when back-clips reach $c/4$. Candidates longer than 200 kb are abandoned. **f**, Copy-number-aware filter, site removal. Inside an interval, each read is scored by its mismatch load R_m , the number of heterozygous sites in the interval at which it carries the alternate allele, and each heterozygous site by $\text{MR} = \bar{R}_m^{\text{ALT}} / (\bar{R}_m^{\text{REF}} + \bar{R}_m^{\text{ALT}})$. At a genuine heterozygous site both allele populations are ordinary reads of the locus and $\text{MR} \approx 0.5$; at a paralogue-specific difference the alternate allele is carried by reads that originate elsewhere and mismatch the reference at many other sites in the same interval, so $\text{MR} \rightarrow 1$. Sites with $\text{MR} \geq 0.7$ are removed from all reads; sites at which either mean is zero are never removed. **g**, Order in which the filters are applied; the copy-number-aware filter (**e,f**) acts last, immediately before graph construction.

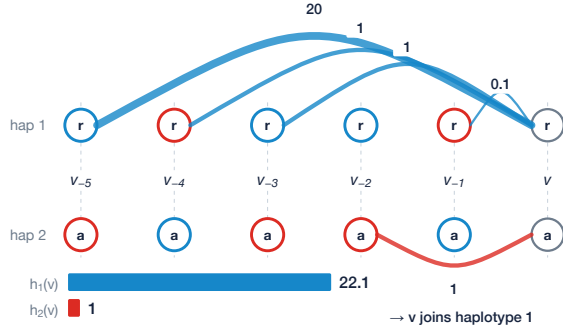
a Allele-pair support between u and v



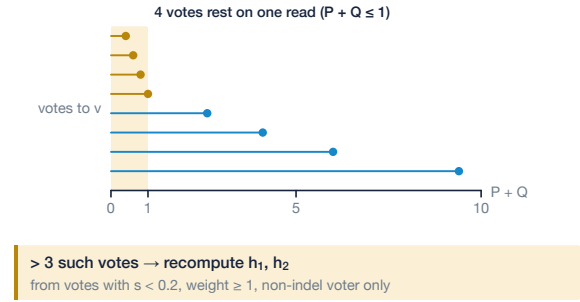
b Vote rule for the pair (u, v)



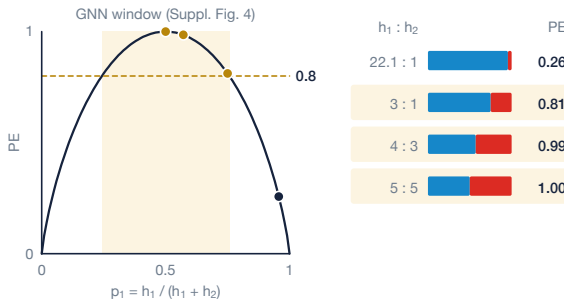
c Weighted votes from up to $k = 35$ upstream variants



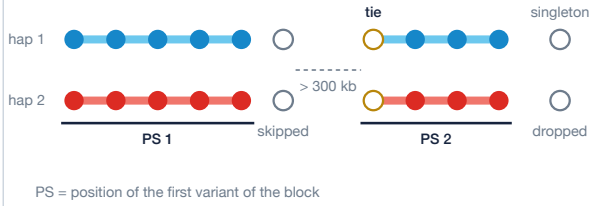
d Single-read guard



e Phasing entropy $PE = -\sum p_i \log_2 p_i$



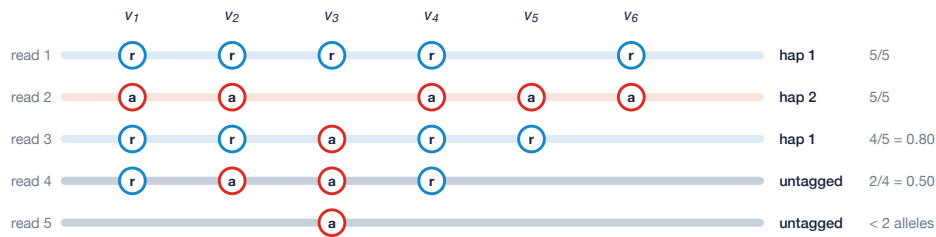
f Block formation



Supplementary Fig. 2: Pairwise allele support, weighted voting and phasing entropy. a,

Between two variants u and v the reads accumulate four weights ($w_{rr}, w_{ra}, w_{ar}, w_{aa}$), each read adding 1 when both bases have quality $\geq Q_{12}$ and 0.1 otherwise. $P = w_{rr} + w_{aa}$ and $Q = w_{ra} + w_{ar}$ are the supports for the two possible pairings and $s = \min(P, Q) / \max(P, Q)$ their ambiguity. **b**, The pair votes for the better-supported pairing: at weight 20 when $s \leq 0.1$ with $P + Q \geq 1$ or, irrespective of s , when one pairing has less than 1 unit of support and the other at least 1; at weight 1 otherwise; and not at all when $s > 0.7$ (0.3 for SNV-5mC pairs, which also cast no vote when $P + Q < 1$). A voting variant that is a tandem-repeat indel votes at weight 0.1. The vote names a haplotype for v relative to the haplotype already assigned to u . **c**, Variant v receives weighted votes from up to $k = 35$ upstream variants that share a read with it, at any distance; line thickness encodes vote weight. The weighted totals h_1 and h_2 decide the haplotype; a tie opens a new phase block. **d**, Guard against a single erroneous ultra-long read: among the votes received by v , those resting on a single read ($P + Q \leq 1$, amber) are counted, and if more than three exist, h_1 and h_2 are recomputed from the votes with $s < 0.2$, weight ≥ 1 and a non-indel voter only. **e**, Phasing entropy of the vote totals, $PE = -\sum p_i \log_2 p_i$ with $p_i = h_i / (h_1 + h_2)$, as a function of the vote split (left) and for four examples (right); values ≥ 0.8 (shaded) open a window for the neural network (Supplementary Fig. 4). PE , h_1 and h_2 are written to the VCF as INFO/PE, INFO/H1 and INFO/H2. **f**, Block formation. Variants are visited in coordinate order; a variant whose next heterozygous variant lies more than 300 kb away ($-\text{distance}$) is skipped and casts no votes, a variant that receives no informative vote or a tie (amber) starts a new phase set, and blocks holding a single variant are dropped (grey). Haplotype labels propagate along the block as the phased genotypes 0|1 and 1|0; the phase set (PS) is the position of its first variant.

a Step 1: read assignment to haplotypes

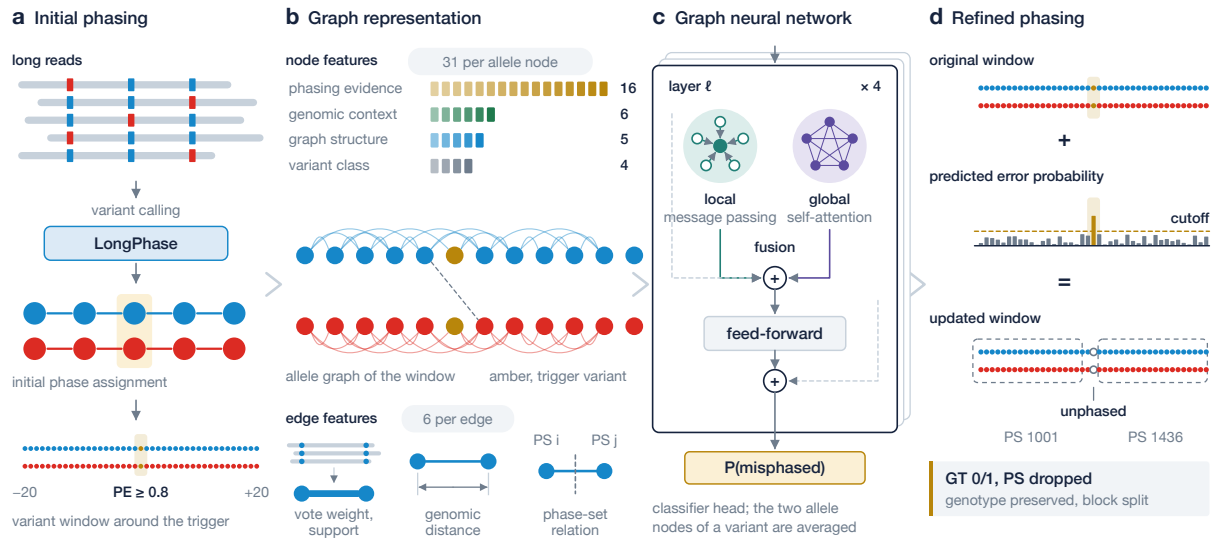


tagged if $\max(n_1, n_2) / (n_1 + n_2) > 0.65$ and weighted evidence > 1
 allele weights: SNV 1, SV 1, indel 0.1, 5mC 0 (so ≥ 2 SNV/SV alleles are needed)

b Step 2: phase re-derivation from tagged reads

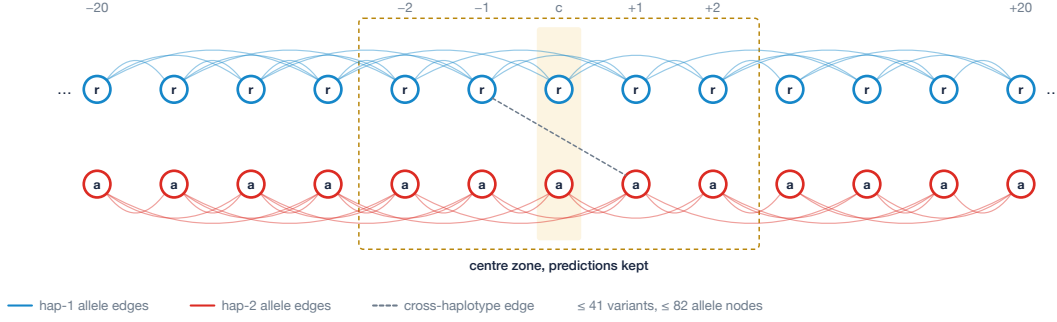


Supplementary Fig. 3: Read-based correction. **a**, Each read is assigned to a haplotype by majority over its phased SNV and SV alleles (indels count 0.1, 5mC 0); reads with majority ≤ 0.65 or fewer than two informative alleles remain untagged. **b**, Each variant's phase is re-derived from the tagged reads and retained only if the consistent fraction ρ exceeds 0.75; otherwise the variant is unphased. Only variants supported by both read populations keep a phase, so assignments driven by a few chimeric or mis-mapped reads are removed. The two thresholds are `-readConfidence 0.65` and `-snpConfidence 0.75`.

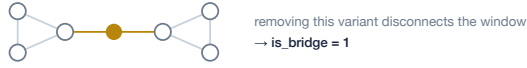


Supplementary Fig. 4: Overview of the graph neural network correction module. **a**, LongPhase phases the called variants from the long-read alignments; every phased variant whose phasing entropy reaches 0.8 (amber) becomes the trigger of a window of ± 20 variants. **b**, The window is encoded as a graph whose nodes are the two alleles of each variant and whose edges are the surviving DOT edges (Supplementary Fig. 5d). Each allele node carries 31 features in four groups and each edge six features (Supplementary Tables 1 and 2); the shaded squares stand for the individual features of a group, not for their values. **c**, Each of four GPS layers passes the graph through a local branch, in which a node attends to its neighbours in the phasing graph (GATv2), and a global branch, in which every allele node in the window attends to every other. The two representations are fused additively with a skip connection from the layer input (dashed grey) and passed through a feed-forward block with a second skip connection. A classifier head then assigns each allele node a probability of being misphased, and the two allele nodes of a variant are averaged. **d**, Variants whose predicted error probability reaches 0.30 are unphased: the genotype is written unphased and without a phase set, the genotype itself is never flipped, and connectivity over the surviving edges splits the phase set in two. Detailed views of the window, the network and the phase-set update are given in Supplementary Figs. 5 and 6.

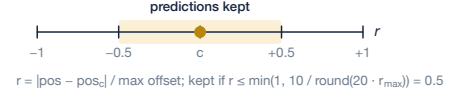
a Window around a trigger variant ($PE \geq 0.8$)



b Bridge-vertex feature



c Centre-zone mask

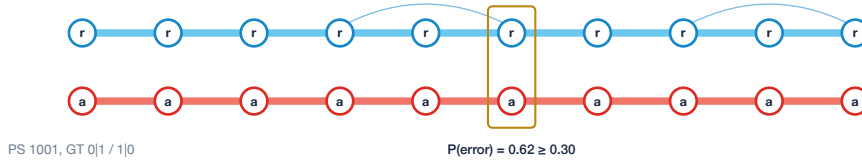


d Exported phasing graph (DOT)

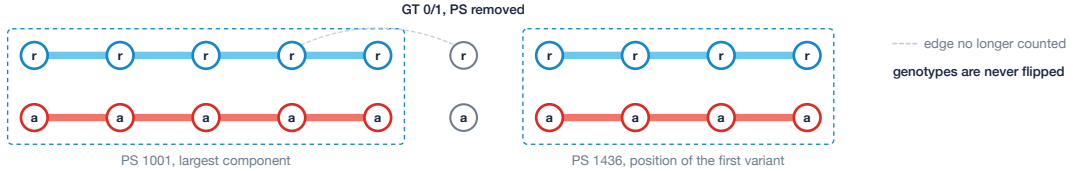
1001.1 → 1543.1 [label=16.0] 1001.2 → 1543.2 [label=1.1]

position.allele → position.allele, label = vote weight cast at the source

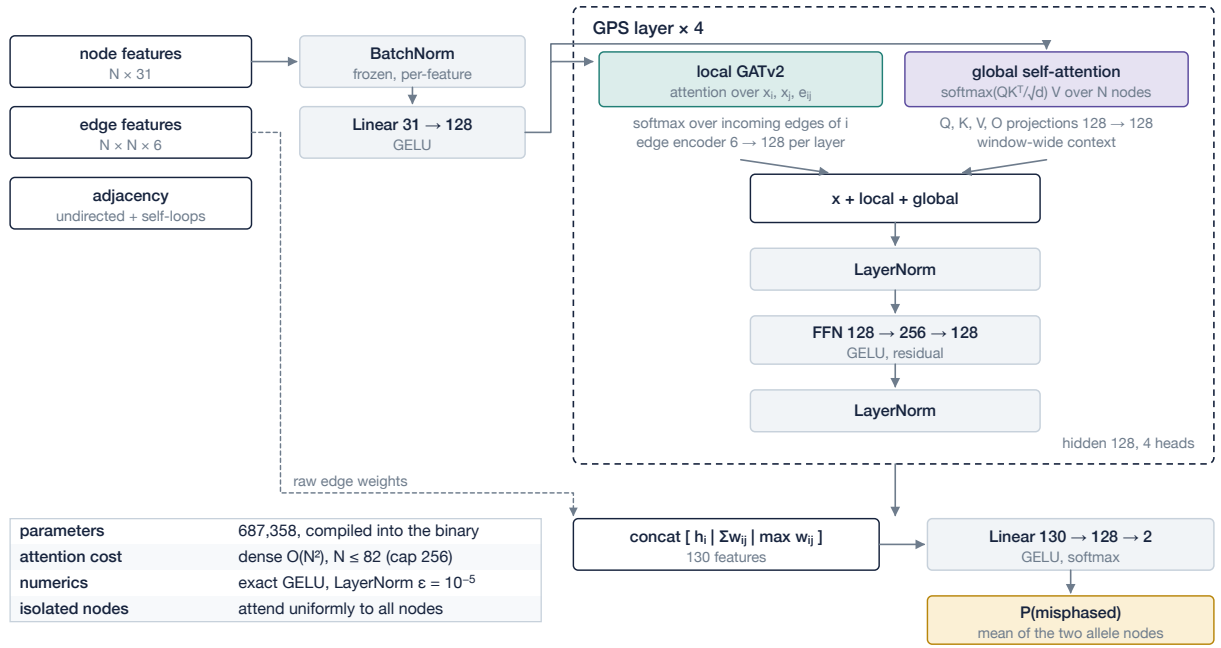
e Before correction: one phase set



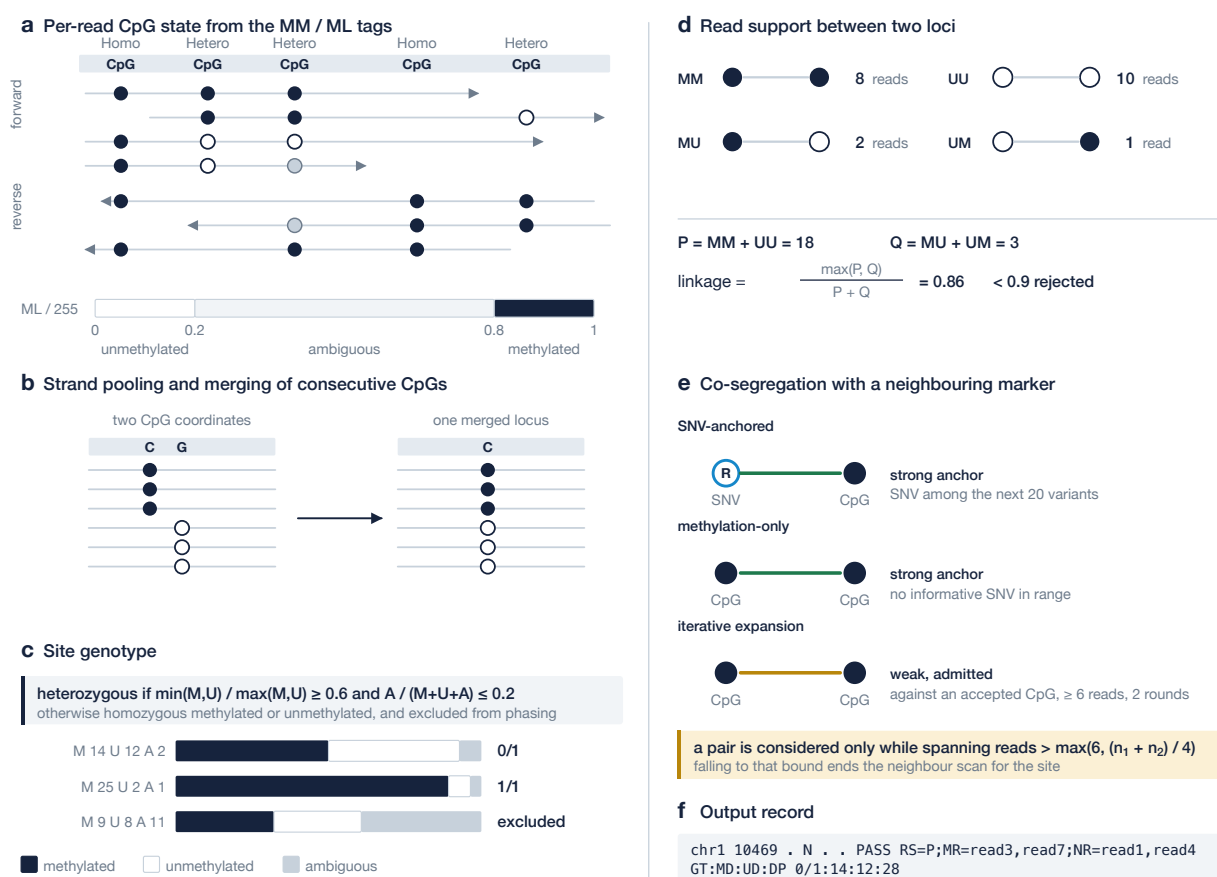
f After correction: variant unphased, phase set split



Supplementary Fig. 5: Window construction and phase-set update for the graph neural network. **a**, A window of ± 20 variants (two allele nodes each) is opened around every phased variant with $PE \geq 0.8$; edges are the DOT edges with both endpoints in the window (blue and red arcs) plus self-loops; a dashed grey edge marks a cross-haplotype link. Predictions are read only in the central zone. **b**, Bridge-vertex feature: a variant whose removal disconnects the window graph. **c**, Definition of the central zone. A window holds at most 41 variants and 82 allele nodes (a 256-node cap exists but is never reached at the default window size); SNV, indel, SV and 5mC records are merged and sorted by position before nodes are created. Because offsets are normalized by the largest offset in the window, $r_{\max} = 1$ in practice and the retained zone is the central half of the window's genomic span. The same rule, with the same constant, defines which labelled nodes carried a training loss, so the zone scored at inference is the zone the network was trained on. Edges are the DOT edges with both endpoints in the window, symmetrized, plus one self-loop per node whose features are the mean over that node's incident edges. The predictions of the two allele nodes of a variant are averaged, and a variant covered by several windows receives the mean over those windows. **d**, The exported phasing graph (DOT) that the module reads: one line per voting pair, source and target written as position.allele, and the vote weight accumulated at the source as the label. **e**, Phase-set update, before correction: a variant with predicted error probability ≥ 0.30 (amber) inside a phase set. **f**, After correction the variant is written as an unphased genotype without PS; connectivity of the remaining variants is recomputed over the surviving DOT edges, the largest component keeps the original PS and every other component receives a new PS equal to the position of its first variant. Genotypes are never flipped.

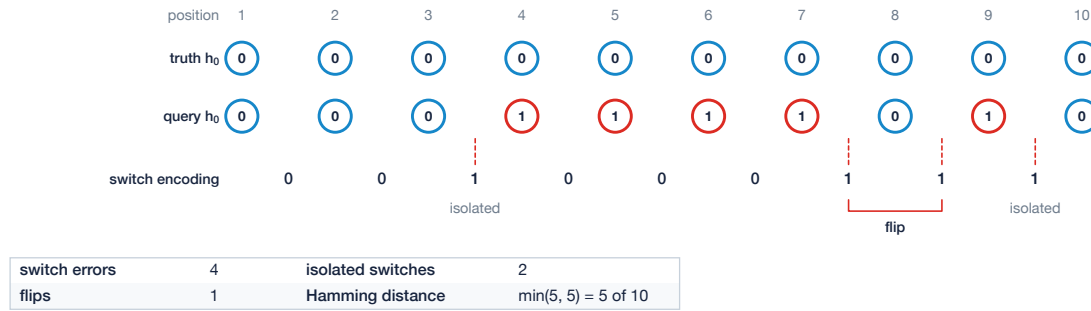


Supplementary Fig. 6: Graph neural network architecture. Node features are standardized and projected to 128 dimensions. Each of four GPS layers combines a local GATv2 branch over the window graph (edge features enter through a per-layer edge encoder) with a global self-attention branch over all nodes, fused additively into the residual stream and followed by layer normalization and a feed-forward block. The final embedding is concatenated with the sum and the non-negative maximum, over the node's incoming edges excluding its self-loop, of the logistic-transformed edge weight $1/(1 + e^{-w})$, and classified into phased-correctly versus misphased; the probabilities of the two allele nodes of a variant are averaged.



Supplementary Fig. 7: Allele-specific methylation calling with modcall. **a**, Per-read CpG state is read from the MM/ML tags on both strands: methylated at $ML/255 \geq 0.8$, unmethylated at ≤ 0.2 , ambiguous in between. Only CpG dinucleotides are considered. **b**, 5mC is called on one strand at a time, so reverse-strand calls are mapped to the forward CpG coordinate and runs of consecutive CpGs are merged into a single locus carrying one read list. This removes strand from the problem and lets **phase** vote on methylation loci with the unmodified graph algorithm. **c**, A site is heterozygous when $\min(M, U) / \max(M, U) \geq 0.6$ and $A / (M + U + A) \leq 0.2$, for methylated, unmethylated and ambiguous read counts M , U and A ; otherwise it is homozygous and excluded. **d**, Balance alone does not imply allele specificity, since stochastic methylation satisfies **c** as well. Read support between two loci is therefore decomposed into the four state pairs, whose concordant and discordant totals are exactly the quantities P and Q that the phasing graph accumulates; a pair is linked when $\max(P, Q) / (P + Q) \geq 0.9$. The example shown falls below the threshold and is rejected. **e**, Linking runs in three modes: against an SNV among the next 20 variants, against another candidate CpG where no informative SNV is in range, and an iterative expansion that admits weak candidates against an already accepted CpG. In every mode a pair is considered only while the reads spanning it exceed $\max(6, (n_1 + n_2) / 4)$, and failing to that bound ends the neighbour scan for that site rather than skipping one pair. **f**, Output record; the MR and NR read names attach the 5mC alleles to reads in **phase**. [Two real-data panels would close the remaining gaps, and both exist in the group's thesis material. (i) Per-site ML probability at heterozygous versus homozygous CpGs with the 0.8 and 0.2 lines marked, showing that it is the homozygous sites that spread across the ambiguous band; the original was computed on 48 sites in the imprinted *SNRPN* locus, too small and too special a sample for a Nature Methods supplement, so recompute genome-wide on HG002. (ii) Distribution of the linkage ratio in **d** stratified by the true class of the site pair (heterozygous–heterozygous, homozygous–homozygous, heterozygous–homozygous). This is the strongest justification available for the 0.9 threshold: in the original the three classes separate at roughly 0.65–1, 0.5–0.75 and 0.5–0.7, showing that the cut removes homozygous sites misclassified as allele-specific rather than being an arbitrary round number. Regenerate both with the submitted version.]

a Switch errors, flips and Hamming distance within one block



b Block statistics



Supplementary Fig. 8: Phasing evaluation metrics. **a**, Within one intersected block, the query haplotype string is compared with the truth in switch encoding; the number of differing positions is the switch-error count, adjacent switch pairs are counted as flips, and the Hamming distance is the minimum over the two orientations of misassigned variants. **b**, Block intersection and the block-level statistics. Only variants that are heterozygous in both files and phased in both are compared (dark bars); blocks left with a single variant are ignored (outline). Phased fraction, block N50 and switch error rate are computed over the intersected blocks as implemented in `longphase compare`.

Placeholder: panels to be regenerated with the submitted version.

Supplementary Fig. 9: Calibration of the voting and read-filtering constants. Each panel is a sweep on HG002 at 10–60 $\times$; the value adopted in LongPhase 2 is marked. **a**, Distribution of pair ambiguity $s = \min(P, Q) / \max(P, Q)$ over all assessed pairs and over the subset that participate in a switch error, showing that both mass and residual error concentrate below $s = 0.1$ and motivating $\alpha = 0.1$ as the boundary of the up-weighted class. **b**, Switch errors and block N50 as a function of the no-vote threshold β . Switch errors fall monotonically as β decreases, so switch error alone does not identify an optimum; $\beta = 0.7$ is the smallest value at which block N50 is not yet degraded, and is adopted on that basis. **c**, Switch errors as a function of the up-weight w applied when $s < \alpha$, at each coverage; the reduction saturates by $w \approx 20$. **d**, Distribution of base quality over all bases and over bases carrying a sequencing error, showing errors concentrated below Q10 and motivating $\theta = 12$; and switch errors as a function of θ . **e**, Switch errors over the grid of single-read-guard parameters N (number of single-read votes that triggers the guard) and μ (ambiguity cut applied to the surviving votes), and switch errors with and without the guard at each coverage. [All five panels exist as figures in the group's thesis material (Po-Hui Tzeng, 2026, Figs. 3.7, 3.10–3.12, 3.14, 3.17, 3.18, 3.21) and should be regenerated with the submitted version on the Clair3 calls and coverages used here, then redrawn to a single consistent style. Two caveats to carry over honestly: in **b** the switch-error curve does not have an interior optimum, so the choice of β rests on the N50 trade-off and must be presented that way; and in **e** the guard slightly *increased* switch errors at 10 $\times$ in the original sweep, which should be shown rather than cropped.]

References

- [1] Rampášek, L. *et al.* Recipe for a general, powerful, scalable graph transformer. In *Advances in Neural Information Processing Systems*, vol. 35 (2022).
- [2] Veličković, P. *et al.* Graph attention networks. In *International Conference on Learning Representations (ICLR)* (2018).
- [3] Brody, S., Alon, U. & Yahav, E. How attentive are graph attention networks? In *International Conference on Learning Representations (ICLR)* (2022).
- [4] Vaswani, A. *et al.* Attention is all you need. In *Advances in Neural Information Processing Systems*, vol. 30 (2017).
- [5] Lin, T.-Y., Goyal, P., Girshick, R., He, K. & Dollár, P. Focal loss for dense object detection. In *IEEE International Conference on Computer Vision (ICCV)* (2017).
- [6] Patterson, M. *et al.* WhatsHap: weighted haplotype assembly for future-generation sequencing reads. *Journal of Computational Biology* **22**, 498–509 (2015).
- [7] Hansen, N. F. *et al.* A complete diploid human genome benchmark for personalized genomics. *Cell* **189**, 4857–4875.e31 (2026).
- [8] Genome in a Bottle Consortium. HG002 T2T-HG002-derived small-variant benchmark (GIAB v5 draft, “v5.0q”). https://ftp-trace.ncbi.nlm.nih.gov/ReferenceSamples/giab/release/AshkenazimTrio/HG002_NA24385_son/T2T-Q100/ (2025). Draft small-variant benchmark projected onto GRCh38 from the T2T-HG002 v1.1 assembly.
- [9] Zheng, Z. *et al.* ClairS: a deep-learning method for long-read tumor–normal pair somatic small variant calling. *Nature Methods* **23**, 1507–1520 (2026).
- [10] Chen, L. *et al.* ClairS-TO: a deep-learning method for long-read tumor-only somatic small variant calling. *Nature Communications* **16** (2025).
- [11] Zheng, Z. *et al.* Clair3-RNA: a deep learning-based small variant caller for long-read RNA sequencing data. *Nature Communications* **16** (2025).
- [12] Zheng, Z. *et al.* Accelerated long-read variant calling with Clair3 for whole-genome sequencing. *Bioinformatics* **42** (2026).
- [13] Kolesnikov, A. *et al.* Local read haplotagging enables accurate long-read small variant calling. *Nature Communications* **15** (2024).
- [14] Keskus, A. G. *et al.* Severus detects somatic structural variation and complex rearrangements in cancer genomes using long-read sequencing. *Nature Biotechnology* **44**, 247–257 (2025).
- [15] Oxford Nanopore Technologies. epi2me-labs/wf-human-variation: Nextflow workflow for the analysis of human genetic variation from Oxford Nanopore sequencing data. <https://github.com/epi2me-labs/wf-human-variation> (2025). Version 2.x; phasing performed with LongPhase (`-use_longphase`).
- [16] Pacific Biosciences. HiFi-somatic-WDL: tumor-normal variant calling workflow using HiFi reads. <https://github.com/PacificBiosciences/HiFi-Somatic-WDL> (2025).
- [17] Cao, S. *et al.* cutehap: Haplotype-Aware structural variant detection in phased Long-Read sequencing data. *Advanced Science* **13** (2026).
- [18] Wang, S. *et al.* A scalable framework for comprehensive typing of polymorphic immune genes from Long-Read data. *Advanced Science* **13** (2026).

- [19] Gabbutt, C. *et al.* Fluctuating DNA methylation tracks cancer evolution at clinical scale. *Nature* **645**, 764–773 (2025).
- [20] O’Neill, K. *et al.* Long-read sequencing of an advanced cancer cohort resolves rearrangements, unravels haplotypes, and reveals methylation landscapes. *Cell Genomics* **4**, 100674 (2024).
- [21] LaFlamme, C. W. *et al.* Diagnostic utility of DNA methylation analysis in genetically unsolved pediatric epilepsies and CHD2 epismutation refinement. *Nature Communications* **15** (2024).
- [22] Iwasaki, Y. *et al.* Bidirectional disruption of *GNAS* transcripts causes broad methylation defects in pseudohypoparathyroidism type 1B. *Proceedings of the National Academy of Sciences* **122** (2025).
- [23] Zalusky, M. P. *et al.* 3-hour genome sequencing and targeted analysis to rapidly assess genetic risk. *Genetics in Medicine Open* **2**, 101833 (2024).
- [24] Smits, D. J. *et al.* Nanopore long-read sequencing for the critically ill facilitates ultrarapid diagnostics and urgent clinical decision making. *European Journal of Human Genetics* **34**, 108–118 (2025).
- [25] Fleming, A. *et al.* Combined approaches, including long-read sequencing, address the diagnostic challenge of HYDIN in primary ciliary dyskinesia. *European Journal of Human Genetics* **32**, 1074–1085 (2024).
- [26] Thomas, R. *et al.* Splice-modulating antisense oligonucleotides targeting a pathogenic intronic variant in adult polyglucosan body disease correct mis-splicing and restore enzyme activity in patient cells. *Nucleic Acids Research* **53** (2025).
- [27] Tian, Y., McDonnell, S. K., Wu, L., Larson, N. B. & Wang, L. Fine mapping regulatory variants by characterizing native CpG methylation with nanopore long-read sequencing. *Human Genetics and Genomics Advances* **7**, 100532 (2026).
- [28] Galvez, J. H. *et al.* Benchmarking of Oxford Nanopore whole genome sequencing for germline variant and CpG methylation detection across Canada’s national platform for genome sequencing and analysis. *Frontiers in Genetics* **17** (2026).
- [29] Flack, N. *et al.* Chromosome-level, nanopore-only genome and allele-specific DNA methylation of Pallas’s cat, *Otocolobus manul*. *NAR Genomics and Bioinformatics* **5** (2023).
- [30] Holt, J. M. *et al.* HiPhase: jointly phasing small, structural, and tandem repeat variants from HiFi sequencing. *Bioinformatics* **40** (2024).
- [31] Battistella, E., Maheshwari, A., Ekim, B., Berger, B. & Popic, V. Graph-based deep reinforcement learning for haplotype assembly with Ralphi. *Genome Research* **35**, 2617–2625 (2025).
- [32] Luo, J., Wang, J., Zhai, H. & Wang, J. GCphase: an SNP phasing method using a graph partition and error correction algorithm. *BMC Bioinformatics* **25** (2024).
- [33] Zhou, Y., Leung, A. W.-S., Ahmed, S. S., Lam, T.-W. & Luo, R. Duet: SNP-assisted structural variant calling and phasing using Oxford nanopore sequencing. *BMC Bioinformatics* **23** (2022).